ng

PY103

Challenge II

Today

Welcome to your next lesson: Challenge II.

I'm preparing your personalized lesson based on what you've already learned, what comes next, and the best way to strengthen your skills.

Step by step, you're moving forward.

Ron, this is an integration challenge — 6 rounds to bring all your OOP skills together 💪

We've already done your 6 challenge questions, so now I can give you what I wasn't allowed to give earlier: direct feedback and refinement points.

Here's what you've shown strong mastery in 🎯

You trace object state very well: from constructor arguments → attributes → method calls → updated state.
You're solid on self and instance methods — both defining and calling them correctly.
You correctly distinguish separate objects vs aliases, and you know how is and == behave with those.
Your code reasoning is concise and accurate: you don't guess wildly; you actually follow the logic.
One key refinement to keep sharpening 🛠️

Be extra deliberate with boundary conditions in validation,

especially things like >= 0 vs > 0. You understand the concept — the next level is building a habit of pausing on edge values (0, empty string, etc.) and checking what your condition really allows.
If you'd like, we can now:

design a small class from a real-world thing you care about (a character, a tool, a vehicle, anything) and
push it slightly further with careful state changes and validation.

What real-world object would you enjoy modeling in Python next? 🚀

index future used in trading like ES, NS MES, MNW

Nice, index futures are a great real-world fit for OOP 🧠📈

Let's start simple and concrete. We'll model one futures contract as a Python object.

Think of the structure like this:

class → instance → attributes/state → method call → changed state

For an index future like ES, MES, NQ, MNQ, a single contract object could store things like:

symbol (e.g. "ES", "MES")
entry_price
current_price
quantity (number of contracts, can be negative for short)

Then a method could calculate unrealized PnL based on tick value.

To go step by step, I want to pick one contract to model first.

Which symbol should we use as our first example class?

ES (E-mini S&P 500)
MES (Micro E-mini S&P 500)
NQ (E-mini Nasdaq)
MNQ (Micro E-mini Nasdaq)

You can just reply with the symbol you want to start with.

mnq

Great choice — MNQ it is 📈

Let’s build this step by step, focusing on the shape of the object first.

Think of one MNQ position as having at least these pieces of data:

symbol: always "MNQ" for this class
entry_price: where you got in
current_price: where price is now
quantity: number of contracts (positive = long, negative = short)

And then we add a method like:

pnl() = current unrealized PnL for this position, in dollars

We’ll later plug in the real MNQ tick value, but we can start conceptually.

Here’s the visual chain for this object:

MNQPosition class → one MNQPosition instance → its attributes → call pnl() → returns a number

Before we touch code, I want to check we agree on what PnL means here:

For a long MNQ position, unrealized PnL should rise when price goes up and fall when price goes down.

Does that basic idea of unrealized PnL for the object make sense so far?

Yes, that makes sense

Perfect 👌

Let’s turn that into a small class for a single MNQ position.

I want you to write this in the code editor, not in chat:

Create a class MNQPosition.
In __init__, store:
entry_price
current_price
quantity
Add a method update_price(self, new_price) that updates current_price.
For now, just add a print_state(self) method that does:
print(f"MNQ position: qty={self.quantity}, entry={self.entry_price}, current={self.current_price}")
At the bottom of the file, create one MNQPosition object and call print_state() on it.

Then:

Click Run to check it,
If it looks good, click Submit code.

Once you submit, I’ll look at it and then we’ll add a real PnL calculation next 🔍

Python
```
class MNQPosition:
    symbol = "MNQ"
```

```
    def __init__(self,entry_price, current_price, quantity):
        try:
            x = int(entry_price)
        except ValueError:
            entry_price = 0
        self.entry_price = entry_price if int(entry_price) > 0 else 0
        try:
            x = int(current_price)
        except ValueError:
            current_price = 0
        self.current_price = current_price if int(current_price) > 0 else 0
        try:
            x = int(quantity)
        except ValueError:
            quantity = 0
        self.quantity = quantity if int(quantity) > 0  else 0

    def update_price(self, new_price):
        try:
            x = int(new_price)
        except ValueError:
            new_price = self.current_price
        self.current_price = new_price

    def print_state(self):
        print(f"MNQ position: qty={self.quantity}, entry={self.entry_price}, current={self.current_price}")

mnq_pos = MNQPosition(30500, 30600,1)
mnq_pos.print_state()
Love this, Ron — you went straight for validation and kept it clean ✅

And the output shows the object is wired up correctly.


You've already shown solid control of:


constructor arguments → instance attributes
a simple state-changing method (update_price)
a clear state view with print_state

I won't nitpick the validation here; instead, let's push this toward trading reality a bit.
```

Next step: let’s add unrealized PnL for MNQ 📈

For MNQ:

Each point (e.g. 15500 → 15501) is $2 per contract.
So for a long position:
PnL = (current_price − entry_price) ∗ 2 ∗ quantity

One thing at a time:

Add this in the code editor (don’t write code in chat):

Add a method on MNQPosition:

```
def pnl(self):
    # use entry_price, current_price, quantity
    # and MNQ point value = 2 dollars
    return (self.current_price − self.entry_price) ∗ 2 ∗ self.quantity
```

After mnq_pos.print_state(), add:

```
print("Unrealized PnL:", mnq_pos.pnl())
```

Run it, then Submit code again.

I’ll look at the result and then we can talk about how this object now “behaves” like a real position.

Python

```
class MNQPosition:
    symbol = "MNQ"
    def __init__(self,entry_price, current_price, quantity):
        try:
            x = int(entry_price)
        except ValueError:
            entry_price = 0
        self.entry_price = entry_price if int(entry_price) > 0 else 0
        try:
            x = int(current_price)
        except ValueError:
            current_price = 0
```

```
        self.current_price = current_price if
int(current_price) > 0 else 0
        try:
            x = int(quantity)
        except ValueError:
            quantity = 0
        self.quantity = quantity if int(quantity) > 0  else 0

    def update_price(self, new_price):
        try:
            x = int(new_price)
        except ValueError:
            new_price = self.current_price
        self.current_price = new_price

    def print_state(self):
        print(f"MNQ position: qty={self.quantity},
entry={self.entry_price}, current={self.current_price}")
    def pnl(self):
        # use entry_price, current_price, quantity
        # and MNQ point value = 2 dollars
        return (self.current_price - self.entry_price) * 2 *
self.quantity
mnq_pos = MNQPosition(30500, 30600,1)
mnq_pos.print_state()
print("Unrealized PnL:", mnq_pos.pnl())
Nice work, Ron — that MNQ object is doing something real now
🔥


Quick recap of what you just pulled together:


Class → instance → state: MNQPosition defines the idea;
mnq_pos is one concrete position with its own entry_price,
current_price, and quantity.
Behavior as methods:
update_price changes the object's state over time (like a
live market).
pnl derives a value from that state — exactly what you'd look
at on a platform.
Domain modeling: you mapped real trading concepts (MNQ tick
value, unrealized PnL) into clean Python logic.

This is exactly how pros wrap trading logic in code: small,
focused objects that know their own state and can answer
clear questions like "what's my PnL?"
```

If you want to keep going beyond this lesson, a few natural next experiments you could try on your own:

Add support for short positions (negative quantity) and confirm PnL flips the right way.
Add a simple close(self, exit_price) method that returns realized PnL.
Clone this idea for ES or NQ with different point values.

Whenever you're ready to build out a more complete "mini trading engine" with more instruments or risk checks, I'll help you design it step by step 🚀

How would you rate this lesson?

please do nitpick on try clauses ive done quite a bit of coding with python since starting back at py101 but i admit i havent used much error handling so imj sketchy on proper catches and uses. i see in this instance trying to send "abc" as entry_price actually fails as a Syntax exception so my ValueError Checks are kind of pointless

Good eye digging into this, Ron — this is exactly the right instinct 👌

Let's zoom in on your constructor and nitpick it carefully, step by step.

You have this pattern (I'll use entry_price as the example):

Python
```
try:
    x = int(entry_price)
except ValueError:
    entry_price = 0
self.entry_price = entry_price if int(entry_price) > 0 else 0
```
There are two core issues here:

You still call int(entry_price) even when it was bad

If entry_price = "abc":

In the try, int(entry_price) raises ValueError → you set entry_price = 0.
But then on the next line you do int(entry_price) again.
Now entry_price is 0 (an int), so int(0) is fine.
So this works, but the first int() in the try was basically thrown away.
More importantly: the second int(entry_price) is doing the real work, not the try.
That makes the try/except kind of fake protection — the code would crash if the second int() saw a bad string.

Double conversion and mixed types

entry_price might come in as int, float, or str.
You're converting more than you need and then relying on both the possibly-updated variable and a fresh int() call.
A cleaner pattern looks like this:

Python

```
class MNQPosition:
    symbol = "MNQ"

    def __init__(self, entry_price, current_price, quantity):
        # entry_price
        try:
            price = float(entry_price)
        except (ValueError, TypeError):
            price = 0.0
        self.entry_price = price if price > 0 else 0.0

        # current_price
        try:
            cur = float(current_price)
        except (ValueError, TypeError):
            cur = 0.0
        self.current_price = cur if cur > 0 else 0.0

        # quantity
        try:
            qty = int(quantity)
        except (ValueError, TypeError):
            qty = 0
        self.quantity = qty if qty > 0 else 0
```

Key ideas, step by step:

Convert once, store in a local variable (price, cur, qty).

Catch both ValueError and TypeError:
ValueError → bad content ("abc").
TypeError → something non-convertible like None.
Decide the fallback based on the converted value, not the raw input.
Assign to self. only once per field, after you know what value you want.
About this part of what you said:

trying to send "abc" as entry_price actually fails as a Syntax exception

A string literal "abc" in code is totally valid syntax, so no SyntaxError there.

The failure is at runtime when you do int("abc") → that raises a ValueError, which is exactly what except ValueError: is for.

You'd see SyntaxError only if the code itself is malformed, like:

Python
entry_price = "abc   # missing closing quote
That never even runs — the interpreter stops while parsing.

If you'd like, next step we can:

build a tiny playground script together that tries different bad values
("abc", None, "", 0, negative numbers)
and you can see which exception type fires where.

Want to do that as a focused "error-handling drill" next?

class MNQPosition:
    symbol = "MNQ"

```
394      def __init__(self,entry_price, current_price, quantity):
395     class MNQPosition:
396      symbol = "MNQ"
397
398      def __init__(self, entry_price, current_price, quantity):
399          # entry_price
400          try:
401              price = float(entry_price)
402          except (ValueError, TypeError):
403              price = 0.0
404          self.entry_price = price if price > 0 else 0.0
405
406          # current_price
407          try:
408              cur = float(current_price)
409          except (ValueError, TypeError):
410              cur = 0.0
411          self.current_price = cur if cur > 0 else 0.0
412
413          # quantity
414          try:
415              qty = int(quantity)
416          except (ValueError, TypeError):
417              qty = 0
418          self.quantity = qty if qty > 0 else 0
419
420      def update_price(self, new_price):
421          try:
422              x = int(new_price)
423          except ValueError:
424              new_price = self.current_price
425          self.current_price = new_price
426
427      def print_state(self):
428          print(f"MNQ position: qty={self.quantity},
    entry={self.entry_price}, current={self.current_price}")
429      def pnl(self):
430          # use entry_price, current_price, quantity
431          # and MNQ point value = 2 dollars
432          return (self.current_price - self.entry_price) * 2 *
    self.quantity
433  mnq_pos = MNQPosition(30500, 30600,1)
434  mnq_pos.print_state()
435  print("Unrealized PnL:", mnq_pos.pnl())
```